App Dev Project Report

1. Student Details

Name: Aman Chawala

Roll Number: 23f3001753

Email: 23f3001753@ds.study.iitm.ac.in

Video Link: https://drive.google.com/file/d/1ZYhZgwQxztfMDnlxlDQu_OJiOrRWa_7m/view?usp=sharing

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Adventure organizations currently rely on spreadsheets, phone calls, and manual coordination to manage trekking activities, which makes it difficult to handle trek approvals, track bookings, avoid overbooking, and maintain trek history. The goal of this project was to design and build a web-based Trekking Management Application that allows an Admin, Trek Staff, and Users (Trekkers) to interact with the system based on their roles, streamlining trek creation, staff assignment, and booking management.

Approach:

The application was built using Flask as the backend framework, organized into Blueprints for authentication, admin, staff, and user functionality, with SQLite as the database and SQLAlchemy as the ORM. A unified User model handles all three roles (Admin, Trek Staff, User) with role-based access control enforced through a custom decorator. Jinja2 templates with Bootstrap 5 were used to build a clean, responsive front end without relying on JavaScript for any core logic. The system was designed around three core entities — User, Trek, and Booking — with business rules enforced in the Flask route handlers to prevent overbooking, restrict trek management to the assigned staff member, and gate access to features by role and account status (e.g., staff approval, blacklisting).

3. AI/LLM Declaration

I used Claude (Anthropic) extensively to help design and implement this application, including the Flask application structure, SQLAlchemy models, route/blueprint logic, role-based access control, and the Jinja2/Bootstrap templates. The extent of AI/LLM usage is substantial — roughly 25-40% of the initial code scaffolding was AI-assisted.

All AI-generated code was reviewed, run, and tested locally by me, including manually verifying the authentication flow, overbooking prevention, role-based access restrictions, and blacklist behavior through direct testing of each route. I can explain the reasoning behind the model relationships, the access-control decorator, and the booking logic, and I made adjustments to the code to fit the assignment's specific requirements.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	Object Relational Mapper (ORM) for the SQLite database
Flask-Login	User authentication and session management
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Werkzeug	Password hashing (generate_password_hash / check_password_hash)

Technology / Library	Purpose
SQLite	Lightweight local database for storing all application data

5. Database Schema / ER Diagram

Tables:

- **User** — stores profile details for all three roles (id, name, email, password_hash, contact, role, is_approved, is_blacklisted, created_at)
- **Trek** — stores trek details created by the Admin (id, name, location, difficulty, duration_days, total_slots, available_slots, assigned_staff_id, status, start_date, end_date, description)
- **Booking** — records a trekker's booking against a trek (id, user_id, trek_id, booking_date, status)

Relationships:

- User (role = trekker) → Booking
- User (role = staff) → Trek (as assigned staff)
- Trek → Booking

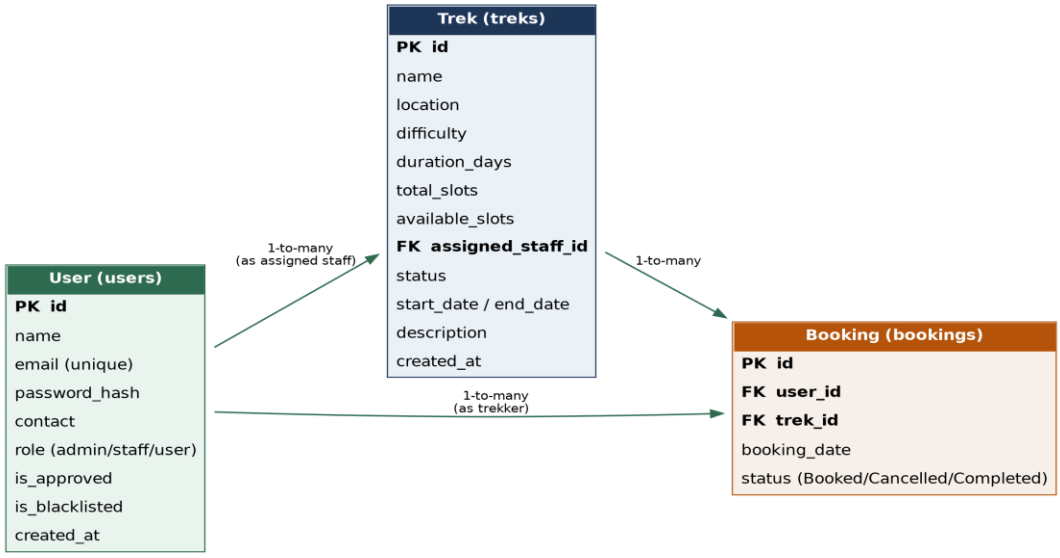

Figure 1: Entity-Relationship diagram of the Trekking Management database

6. API Resource Endpoints

The application exposes the following server-rendered routes (grouped by role), which act as the resource endpoints for the system:

Endpoint	Method	Description
/auth/login	GET / POST	Authenticate a user and start a session
/auth/register	GET / POST	Self-register as Trek Staff or User (Trekker)
/auth/logout	GET	End the current session
/admin/dashboard	GET	Admin stats: total treks, users, staff, bookings
/admin/treks	GET	List all treks

Endpoint	Method	Description
/admin/treks/new	GET / POST	Create a new trek
/admin/treks/<id>/edit	GET / POST	Edit an existing trek
/admin/treks/<id>/delete	POST	Delete a trek
/admin/treks/<id>/assign	POST	Assign / unassign staff to a trek
/admin/staff	GET	List all trek staff
/admin/staff/<id>/approve	POST	Approve a staff registration
/admin/staff/<id>/toggle-blacklist	POST	Blacklist / reinstate a staff member
/admin/users	GET	List all trekkers
/admin/users/<id>/toggle-blacklist	POST	Blacklist / reinstate a trekker
/admin/bookings	GET	View all bookings system-wide
/admin/search	GET	Search treks, staff, or users by name/ID
/staff/dashboard	GET	View treks assigned to the logged-in staff
/staff/treks/<id>	GET	View trek detail and registered participants
/staff/treks/<id>/update	POST	Update available slots and trek status
/user/dashboard	GET	Trekker dashboard: open treks and active bookings
/user/treks	GET	Browse / search / filter treks
/user/treks/<id>/book	POST	Book a trek (subject to slot & status checks)
/user/bookings/<id>/cancel	POST	Cancel an active booking
/user/history	GET	View full booking history
/user/profile	GET / POST	View / edit profile details

7. Architecture and Features

Architecture Overview:

- **app.py** — application factory, blueprint registration, admin seeding, login manager setup
- **config.py** — app configuration (secret key, SQLite database URI)
- **extensions.py** — shared SQLAlchemy and Flask-Login instances
- **utils.py** — role_required decorator for role-based access control
- **models.py** — User, Trek, and Booking SQLAlchemy models
- **/routes** — Flask Blueprints: auth.py, admin.py, staff.py, user.py
- **/templates** — Jinja2 HTML templates organized by role (admin/, staff/, user/, auth/) plus a shared base.html
- **/static/css** — custom CSS built on top of Bootstrap 5

Implemented Features:

- **Role-based registration and login** for Admin, Trek Staff, and Trekkers, with staff requiring admin approval before login
- **Full trek CRUD** (name, location, difficulty, duration, slots, dates, status)
- **Staff approval and blacklist management**

- **Assigning staff** to a trek, restricted to approved and non-blacklisted staff
- **Prevention of overbooking** beyond available slots, and trek management restricted to the assigned staff member only
- **Trek search and filtering** by name, location, and keyword
- **Booking and cancellation** with automatic slot release on cancellation
- **Complete booking history** for both admins (all bookings) and trekkers (personal history)
- **Admin search** of treks, staff, and users by name or ID
- **Secure password hashing** using Werkzeug's generate_password_hash / check_password_hash
- **Responsive UI** built with Bootstrap 5

Additional Features:

- **Admin analytics dashboard** showing total treks, users, staff, bookings, and pending staff approvals at a glance
- **Account status gating** preventing blacklisted or unapproved accounts from accessing the system
- **Status lifecycle management** (Booked / Cancelled / Completed) tracked per booking, and trek status (Pending / Approved / Open / Closed / Completed) tracked per trek

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1ZYhZgwQxztfMDnlxlDQu_OJiOrRWa_7m/view?usp=sharing